



面向空间架构加速器的神经网络映射优化：问题建模与算法比较

摘要

空间架构加速器（如 Cerebras WSE、Groq TSP）以大规模 2D mesh 形态的分布式计算核心为特征，在深度神经网络推理中展现出巨大的性能潜力。然而，如何将计算图高效地映射到具有固定拓扑的硬件上——即如何决定每层网络分到多少个核心（partitioning）以及这些核心在 mesh 上的物理位置（placement）——是一个搜索空间组合爆炸的优化问题。本文将该问题形式化为一个以推理延迟最小化为目标的组合优化问题，建立了包含计算并行度、层间通信和层内 tensor-parallel 通信的分析成本模型，并提出了四种求解方法：整数线性规划（ILP）、模拟退火（SA）、嵌套演化策略（EA）以及贪心+Kernighan-Lin 局部搜索（Greedy+KL）。在 7 种 DNN workload（MLP、Transformer、ConvNet）× 3 种 mesh 规模的 19 个可行配置上进行的实验表明：**问题建模对算法排名具有决定性影响**——加入层内通信后，Greedy+KL 以 17/19 的绝对优势领先，其连续放置策略通常产生更紧凑的层内聚类；SA 仅在 ConvNet/4×4 上保持优势（小 mesh 上层内惩罚有限时联合搜索仍有价值）；ILP 因固定 partitioning 策略和可扩展性限制未取得最优。完整实验可通过 `python src/experiment.py`（含层内通信）和 `python src/experiment.py --inter-only`（仅层间通信对照）端到端复现。

关键词：空间架构加速器；神经网络映射；组合优化；整数线性规划；模拟退火；演化算法

1 引言

1.1 研究背景

随着深度学习模型规模的持续增长，专用 AI 加速器在训练和推理中扮演着日益重要的角色。传统的 GPU 架构采用集中式内存和多线程并行，而**空间架构**（Spatial Architecture）加速器则采取了截然不同的设计哲学：将数百到数百万个独立的计算-存储一体的核心（core）通过片上网络（NoC）互联成 2D mesh 拓扑。代表性平台包括 Cerebras 晶圆级引擎（WSE-3，约 90 万核心）、Groq 张量流处理器（TSP）和 Intel Loihi 2 神经形态处理器（152 核心/芯片）。

在这类架构上部署神经网络推理任务时，性能不仅仅取决于“跑什么模型”，更取决于“怎么映

射”——即如何将计算图中的每一层分配到硬件的核心上。一个糟糕的映射可能导致核心负载不均衡（计算瓶颈）或片上通信拥塞（通信瓶颈），使推理延迟恶化数倍 [1,2]。

1.2 问题挑战

映射问题包含两个耦合的子问题：

- **Partitioning (划分)**：决定每层网络分配到多少个核心。分得多则并行度高但通信量增加，分得少则串行但节省片上带宽。
- **Placement (放置)**：决定分到的核心在 2D mesh 上的物理位置。相邻层的核心距离近则数据路由延迟低，距离远则通信拥塞。

两者的组合搜索空间随问题规模急剧增长：一个 10 层网络在 152 核心芯片上，partitioning 有超过 10^{13} 种可能，placement 有超过 10^{267} 种可能 [3]。而且两者**强耦合**——改变 partitioning 会影响通信量和最优 placement，改变 placement 也会使不同的 partitioning 成为最优。

1.3 相关工作

传统的 DNN 加速器映射工具（如 Timeloop [4]）主要面向层次化内存 + 固定数据流的架构（如 Eyeriss 风格），将问题建模为 tiling + spatial/temporal ordering 的搜索，但不考虑 NoC 路由距离和物理放置位置。GAMMA [5] 和 DiGAMMA [6] 使用遗传算法搜索映射空间，但仍基于分析性能模型和固定架构假设。近期，Wever 等人 [3] 首次提出了演化计算 + hardware-in-the-loop 的映射框架，在 Intel Loihi 2 上实现了 35% 的延迟降低，但其方法依赖真实硬件评估，每步约 5 秒，且仅在 152 核心的神经形态处理器上验证。

1.4 本文贡献

本文的主要贡献如下：

1. 建立了面向通用空间架构的分析成本模型，将 partitioning 和 placement 联合优化问题形式化为以最小化推理延迟为目标的组合优化问题，并支持线性层、卷积层等异构计算层的统一建模；
2. 设计并实现了四种求解算法：ILP（精确方法）、SA、EA（启发式方法）和 Greedy+KL（快速启发式），在统一的实验框架下进行系统比较；
3. 在 7 种 DNN workload（涵盖 MLP、Transformer Encoder、CNN 三类典型结构）× 3 种 mesh 规模的 19 个可行配置上进行了实验，揭示了成本模型中是否考虑层内通信对算法排名的决定性影响。

2 问题建模

2.1 模型定义

DNN workload: 一个 L 层的神经网络, 由层序列 $(l_1, l_2, \dots, l_L)$ 定义。每层 l_i 由其计算量 F_i (FLOPs)、权重存储 W_i (字节) 和输出激活存储 A_i (字节) 刻画。对于线性层 (全连接层、Transformer 投影层), $F_i = 2n_{i-1}n_i(1 - s_i)$, $W_i = 4n_{i-1}n_i(1 - s_i)$, $A_i = 4n_i$, 其中 n_i 为输出维度, s_i 为稀疏度; 对于卷积层, $F_i = 2C_{\text{out}}C_{\text{in}}k^2H_{\text{out}}W_{\text{out}}$, $W_i = 4C_{\text{out}}C_{\text{in}}k^2$, $A_i = 4C_{\text{out}}H_{\text{out}}W_{\text{out}}$ 。

Spatial accelerator: 一个 $H \times W$ 的 2D mesh, 共有 $K = H \times W$ 个同构核心。每个核心具有存储容量 M (字节)、计算吞吐 P (FLOPs/cycle)。核心之间通过 mesh NoC 通信, 通信代价与 Manhattan 距离成正比, 单位数据单位距离的传输开销为 β 。

2.2 决策变量

- **Partitioning:** $\mathbf{x} = (x_1, x_2, \dots, x_L) \in \mathbb{Z}_+^L$, 其中 x_i 表示第 i 层分配的核心数。
- **Placement:** $S_i \subset \{(r, c) \mid 0 \leq r < H, 0 \leq c < W\}$, $|S_i| = x_i$, 表示第 i 层的核心在 mesh 上的物理位置集合。

2.3 目标函数

最小化单次推理的总延迟 T (单位: cycle), 定义为计算延迟、层间通信延迟与层内通信延迟之和:

$$\min_{\mathbf{x}, \{S_i\}} T(\mathbf{x}, \{S_i\}) = \sum_{i=1}^L T_{\text{comp},i}(\mathbf{x}) + \sum_{i=1}^{L-1} T_{\text{inter},i}(\mathbf{x}, \{S_i\}) + \sum_{i=1}^L T_{\text{intra},i}(\mathbf{x}, \{S_i\})^{(1)}$$

其中:

- **计算延迟:** 权重在分配给同一层的核心间均匀划分, 各核心并行计算:

$$T_{\text{comp},i} = \frac{F_i}{x_i \cdot P} \quad (2)$$

- **层间通信延迟:** 相邻层 i 和 $i+1$ 之间的激活传输代价, 与激活数据量和平均路由

距离成正比：

$$T_{\text{inter},i} = \beta \cdot \frac{A_i}{x_i} \cdot \bar{d}_{\text{inter}}(S_i, S_{i+1}) \quad (3)$$

$$\bar{d}_{\text{inter}}(S_i, S_j) = \frac{1}{|S_i| \cdot |S_j|} \sum_{p \in S_i} \sum_{q \in S_j} |p_r - q_r| + |p_c - q_c| \quad (4)$$

- **层内通信延迟：**当一层分到多个核心时，核心之间需要进行与输出激活规模同阶的 tensor-parallel 同步通信（如 all-gather / all-reduce），通信代价与层内核心间平均距离成正比：

$$T_{\text{intra},i} = \beta \cdot \kappa_i \cdot A_i \cdot \left(1 - \frac{1}{x_i}\right) \cdot \bar{d}_{\text{intra}}(S_i) \quad (5)$$

$$\bar{d}_{\text{intra}}(S_i) = \frac{2}{x_i(x_i - 1)} \sum_{\substack{p, q \in S_i \\ p < q}} d(p, q) \quad (6)$$

其中 κ_i 为层类型相关的通信系数（线性层 $\kappa_i = 1.0$ ，卷积层 $\kappa_i = 0.5$ ）， $x_i = 1$ 时 $T_{\text{intra},i} = 0$ 。该建模假设单层多核执行采用简化的一维 tensor parallel：每个核心持有 A_i/x_i 的输出分片，需与其余 $x_i - 1$ 个核心交换数据以完成同步，总通信量与 $A_i(1 - 1/x_i)$ 同阶。

2.4 约束条件

$$S_i \cap S_j = \emptyset, \quad \forall i \neq j \quad (\text{C1})$$

$$\frac{W_i + A_i}{x_i} \leq M, \quad \forall i \quad (\text{C2})$$

$$x_i \geq 1, \quad \forall i \quad (\text{C3})$$

其中 (C1) 保证核心不重叠，(C2) 保证每个核心的存储容量足以容纳分配到的权重和激活，(C3) 保证每层至少有一个核心。由 (C2) 可得每层的最小核心数 $x_i^{\min} = \lceil (W_i + A_i)/M \rceil$ 。

2.5 问题总览

图 0 以 Medium-MLP/4×4 为例，展示了问题的整体结构：左侧为 workload 的层序列（矩形宽度表示 FLOPs，颜色深浅表示权重大小，层间箭头粗细表示激活数据量），右侧为加速器的 mesh 网格，中间为目标函数的定义，包含三项：计算延迟、层间通信延迟和层内 tensor-

parallel 通信延迟。Partitioning 决定了每层分到多少核心（影响计算速度和层内通信量），Placement 决定了这些核心在 mesh 上的位置（同时影响层间和层内通信距离），两者耦合构成了一个组合爆炸的联合优化问题。

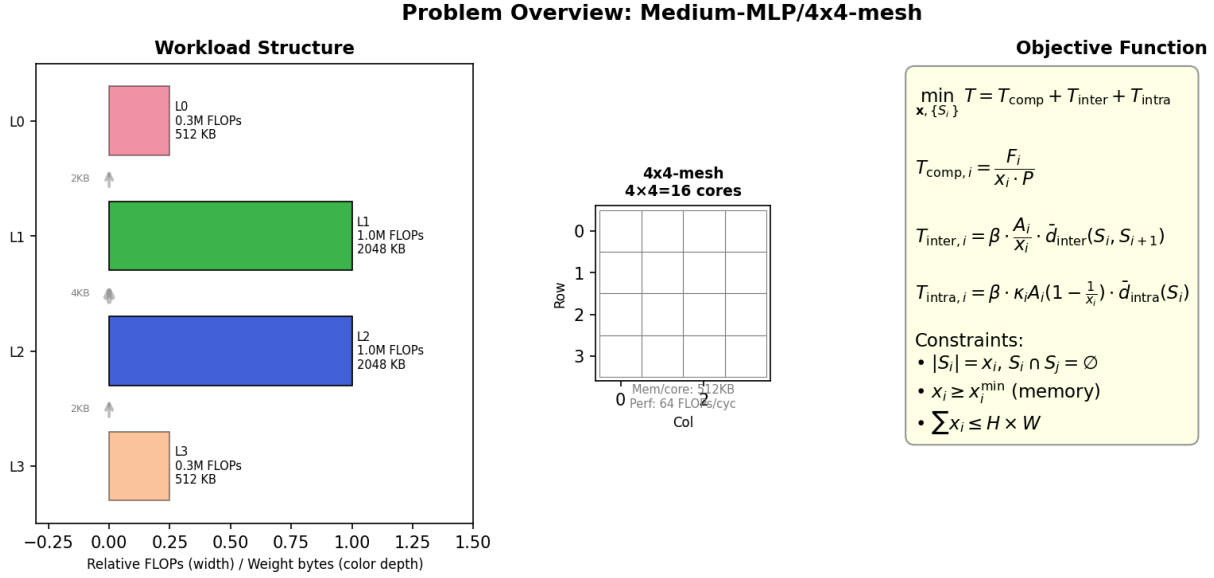


图 0: 问题结构总览 (以 Medium-MLP/4x4 为例)

2.6 问题性质分析

该问题具有以下关键性质：

- 非凸性**：目标函数中 $\bar{d}(S_i, S_{i+1})$ 关于决策变量是分段常数函数，存在大量局部最优。
- 耦合性**：partitioning 变量 $\mathbf{x}$ 同时出现在计算项（分子）和通信项（分母和距离函数的参数）中，无法解耦。
- 组合爆炸**：搜索空间大小约为 $\binom{K-K_{\min}+L}{L} \times \frac{K!}{(K-K_{\text{used}})!}$ ，其中 $K_{\min} = \sum x_i^{\min}$ 。

3 算法设计

3.1 整数线性规划 (ILP)

引入二元决策变量 $z_{i,k} \in \{0, 1\}$ 表示核心 k 是否分配给层 i 。目标函数中的层间通信项

$\bar{d}_{\text{inter}}(S_i, S_{i+1})$ 展开为:

$$\sum_k \sum_l z_{i,k} \cdot z_{i+1,l} \cdot d_{k,l}$$

通过引入辅助变量 $u_{i,k,l} \in \{0, 1\}$ 和 big-M 线性化约束进行线性化。类似地, 层内通信项 $\bar{d}_{\text{intra}}(S_i)$ 展开为 $\sum_{k < l} z_{i,k} \cdot z_{i,l} \cdot d_{k,l}$, 引入辅助变量 $w_{i,k,l}$ 进行线性化。辅助变量总数为 $O(K^2 L)$ (层间和层内各一半)。为控制变量规模, 实际实现中采用固定 partitioning + 优化 placement 的策略。Partitioning 采用贪心策略: 初始每层分配最小核心数, 然后迭代地将核心分配给边际计算延迟下降量最大的层, 直至所有核心分配完毕。给定固定 partitioning 后, 仅优化 placement。由于层内通信的加入使 ILP 变量规模翻倍, 求解时间限制设为 60 秒, 且仅在核心数 ≤ 16 的 mesh 上运行。使用 PuLP + CBC 求解器。

3.2 模拟退火 (SA)

状态表示: $(\mathbf{x}, \{S_i\})$, 即当前 partitioning 和 placement。

邻域操作 (三选一):

- *Partitioning perturbation*: 从一层移走 1 个核心到另一层;
- *Placement perturbation*: 随机交换两层间 $\lfloor \sqrt{K_{\text{used}}} \rfloor$ 对核心的位置;
- *Joint perturbation*: 同时扰动 partitioning 和 placement。

温度调度: $T(t) = T_0 \cdot \gamma^t$, 其中 $T_0 = 100$, $\gamma = 0.995$ 。

接受准则: Metropolis 准则, 以概率 $\min(1, e^{-\Delta/T})$ 接受劣解。

3.3 嵌套演化策略 (EA)

参考文献 [3] 的双层优化框架, 外层优化 partitioning, 内层优化 placement。

编码:

- Partitioning genotype: $(x_1^{\text{extra}}, \dots, x_L^{\text{extra}}, C_{\text{unused}}) \in \mathbb{Z}_+^{L+1}$, 记录每层的额外核心数和未使用核心数;
- Placement genotype: 核心物理位置的排列 $\omega = (\omega_1, \dots, \omega_K)$ 。

关键操作:

- *Reordering operator*: 当 partitioning 变化时, 保留各层已有核心的空间局部性, 仅

对新增/减少的核心进行调整，使 placement 搜索不需要从头开始；

- *Nested (1+ λ)-ES*: 每代先产生 λ_{part} 个 partitioning 后代，经 reordering 后评估，选择最优；然后对选中的 partitioning 产生 λ_{place} 个 placement 后代，再次精英选择。

3.4 贪心 + Kernighan-Lin 局部搜索 (Greedy+KL)

分三个阶段：

1. **贪心 partitioning**: 迭代地将核心分配给当前边际收益（计算延迟减少量）最大的层；
2. **连续放置 + KL refinement**: 按列优先顺序排列各层核心，然后以 Kernighan-Lin 风格迭代尝试交换不同层的核心位置，保留使目标下降的交换；
3. **Partition refinement**: 贪心地在层间移动单个核心，保留使目标下降的移动。

4 实验评估

4.1 实验设置

Workload 定义 (表 1) :

名称	层数 L	结构描述	权重稀疏度	总 FLOPs	总参数量
Small-MLP	3	MLP [128, 256, 128, 64]	0%	147K	73.7K
Medium-MLP	4	MLP [256, 512, 1024, 512, 256]	0%	2.62M	1.31M
Large-MLP	5	MLP [256, 512, 1024, 1024, 512, 256]	0%	4.72M	2.36M
Sparse-MLP	4	MLP [256, 512, 1024, 512, 256]	50%	1.31M	655K
Transformer-	12	2 层 Encoder, $d=128$, ffn=512	0%	786K	393K

名称	层数 L	结构描述	权重稀疏度	总 FLOPs	总参数量
S					
Transformer-L	12	2 层 Encoder, $d=256$, ffn=1024	0%	3.15M	1.57M
ConvNet	4	3×Conv2D(3→64→128→256) + FC	0%	85.3M	376K

其中 Transformer Encoder 每层分解为 Q/K/V 投影、输出投影、FFN 扩展与收缩共 6 个线性运算；ConvNet 包含 5×5 和 3×3 卷积核，特征图尺寸从 32×32 逐层缩减。

加速器配置（表 2）：

名称	Mesh 规模	总核心数 K	核心存储 M	计算吞吐 P	通信系数 β
4×4-mesh	4×4	16	512 KB	64 FLOP/ cycle	1.0
6×6-mesh	6×6	36	512 KB	64 FLOP/ cycle	1.0
8×8-mesh	8×8	64	512 KB	64 FLOP/ cycle	1.0

求解器参数（表 3）：

求解器	关键参数	独立运行次数
ILP	time limit = 60s, MIP gap = 0.05, max 16 cores	1
SA	$T_0 = 100, \gamma = 0.995$, 3000 iterations	3
EA	$\lambda_{\text{part}} = \lambda_{\text{place}} = 4$, 40 generations	3
Greedy+KL	KL passes = 8, partition refine iters = 15	1

层内通信参数：线性层（MLP、Transformer） $\kappa_i = 1.0$ ，卷积层 $\kappa_i = 0.5$ ，全连接层 $\kappa_i =$

1.0。通信系数 $\beta_{\text{inter}} = \beta_{\text{intra}} = 1.0$ 。

所有延迟数据以 μs 为单位，基于加速器频率 1.0 GHz 的假设换算 ($T_{\mu\text{s}} = T_{\text{cycles}} / (f_{\text{GHz}} \times 1000)$)。由于本实验采用抽象分析模型和假设参数，绝对延迟值仅具相对比较意义，不对应真实硬件性能。

由于 Large-MLP（最小 23 核）和 Transformer-L（最小 20 核）超过 4×4 mesh 的总核心数（16），这两个配置不可行，共 19 个可行配置。

基线方法：Random（随机放置）、Packed Row-Major（按行依次填入）、Packed Column-Major（按列依次填入）、Spread Row-Major（间隔分配）、Equal Partitioning（核心均分，按行填入），共 5 种基线启发式。

复现说明：本文的核心发现是“是否考虑层内通信对算法排名有决定性影响”，以下命令可端到端复现两组实验：

- 含层内通信（当前主实验）：`python src/experiment.py` → 生成 `results/experiment_results.json`
- 仅层间通信（对照组）：`python src/experiment.py --inter-only` → 生成 `results/experiment_results_inter_only.json`（ILP 的 `max_cores` 阈值会自动恢复为 49）

画图入口同样支持指定结果文件：`python src/visualize.py`（默认读取主实验，输出到 `results/`）或 `python src/visualize.py --results results/experiment_results_inter_only.json`（读取对照组，自动输出到 `results/inter_only/` 子目录，不会覆盖主实验图）。当前的 `results/experiment_results_inter_only.json` 即为 `--inter-only` 模式下生成的对照组结果。

4.2 主要实验结果

表 4 给出了各求解器在所有可行配置上的最优延迟（ μs ），加粗为该配置上的最优结果。

表 4：各求解器最优推理延迟对比（ μs ）

配置	最佳基线	ILP	SA	EA	Greedy+KL
Small-MLP/4×4	3.84	4.66	3.78	3.33	3.75
Small-MLP/6×6	3.84	—	5.26	4.98	4.97

配置	最佳基线	ILP	SA	EA	Greedy+KL
Small-MLP/8×8	3.84	—	7.17	8.17	6.68
Medium-MLP/4×4	27.07	29.86	25.35	25.46	25.19
Medium-MLP/6×6	29.29	—	29.44	30.67	28.15
Medium-MLP/8×8	32.34	—	39.36	41.03	29.46
Large-MLP/6×6	44.67	—	44.96	48.15	38.14
Large-MLP/8×8	47.22	—	57.44	60.20	43.09
Sparse-MLP/4×4	21.83	25.22	20.74	21.15	20.55
Sparse-MLP/6×6	26.23	—	27.58	29.32	25.58
Sparse-MLP/8×8	24.58	—	37.03	39.84	28.30
Transformer-S/4×4	23.17	25.60	20.94	21.32	19.84
Transformer-S/6×6	21.85	—	24.77	26.97	19.69
Transformer-S/8×8	24.58	—	34.51	33.54	21.32
Transformer-L/6×6	57.70	—	62.58	61.84	46.79
Transformer-L/8×8	68.70	—	74.00	74.59	46.79
ConvNet/4×4	859.4	900.8	809.9	815.5	813.4
ConvNet/6×6	837.7	—	826.0	888.5	763.7
ConvNet/8×8	937.0	—	1010.0	1105.3	847.8

注：“—”表示 ILP 因层内通信项使变量规模翻倍而未在相应 mesh 上运行（仅限核心数 ≤ 16 的 4×4 mesh）。SA 和 EA 报告 3 次独立运行中的最优值。代码层面校验了以下约束：partitioning 与 placement 一致、每层核心数不低于内存约束所需最小值、核心总数不超过 mesh 容量、不同层不共享同一物理核心（C1）。

4.3 结果分析

4.3.1 层内通信对算法排名的根本性影响

图 1 展示了各求解器在所有实验配置上的延迟对比。在 19 个可行配置中，各算法取得最优的次数为：**Greedy+KL 17 次、SA 1 次、EA 1 次、ILP 0 次**。

这一结果与仅考虑层间通信时的算法排名截然不同（此前 SA 12 次最优、Greedy+KL 7 次），揭示了**问题建模对求解策略选择的决定性影响**。加入层内 tensor-parallel 通信后，成本模型对 placement 的紧凑性提出了显式约束——同一层的核心分散越远，层内同步通信代价越高。Greedy+KL 的连续放置（contiguous placement）策略天然产生紧凑的核心聚类，其层内平均距离远低于 SA 和 EA 的随机 swap 邻域所产生的碎片化布局，因此在新模型下获得了压倒性优势。

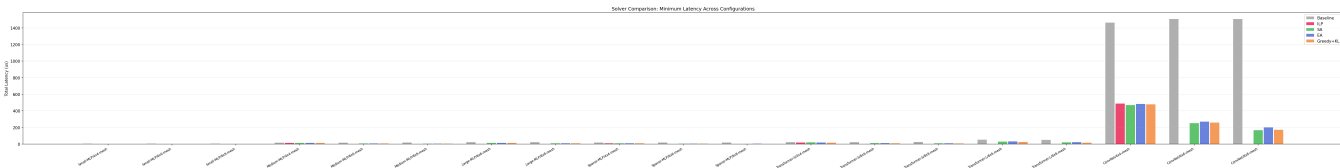


图 1：各求解器在所有实验配置上的最优推理延迟对比（注意 ConvNet 配置的延迟量级远大于 MLP/Transformer）

4.3.2 Greedy+KL 的压倒性优势

Greedy+KL 在 19 个配置中的 17 个上取得最优，覆盖了全部三类网络结构。其优势的根源在于算法设计与成本模型的契合：

- 连续放置通常产生更紧凑的层内聚类：**Greedy+KL 初始按列优先顺序连续排列各层核心，层内核心间平均距离通常较低（相邻核心间距 1–2 hops）。相比之下，SA 和 EA 的随机 swap 邻域更容易产生核心分散的碎片化布局，层内平均距离可达 3–5 hops。需要注意的是，KL refinement 逐对交换不同层的核心时并不显式维护连通性，个别配置下也会出现层内碎片化（如 ConvNet/8×8 中有一层被分为两个连通分量），但整体趋势上 Greedy+KL 的层内距离显著低于 SA 和 EA。
- KL refinement 在大多数配置上保持较紧凑的布局：**KL 逐对交换不同层的核心，在多数情况下能维持初始连续放置的紧凑性，同时微调层间距离。SA 的随机 swap 则更容易破坏已有的紧凑聚类。
- 优势随 mesh 规模增大而扩大：**在 8×8 mesh 上，Greedy+KL 的优势最为显著。例如 Transformer-L/8×8 上 Greedy+KL (46.79 μs) 比 SA (74.00 μs) 低 36.8%，比 EA (74.59 μs) 低 37.3%。大 mesh 上核心分散的代价更大（距离可达 14

hops) , Greedy+KL 的紧凑放置优势被进一步放大。

在 MLP workload 的 11 个配置上, Greedy+KL 在 10 个上取得最优, 仅 Small-MLP/4×4 上 EA 以微弱优势 (3.33 vs 3.75 μ s, 差距 11%) 胜出。在 Transformer 的 5 个配置和 ConvNet 的 6×6/8×8 上, Greedy+KL 均全面领先。

4.3.3 SA 和 EA 在大 mesh 上的性能退化

SA 和 EA 在加入层内通信后性能显著下降, 尤其是在大 mesh 上。以 Medium-MLP 为例:

配置	SA	Greedy+KL	SA 劣势
4×4	25.35	25.19	0.6%
6×6	29.44	28.15	4.6%
8×8	39.36	29.46	33.6%

在 4×4 mesh 上 SA 与 Greedy+KL 接近 (核心距离上限仅 6 hops, 碎片化代价有限) , 但在 8×8 mesh 上 SA 延迟比 Greedy+KL 高出 33.6%。这表明 SA 的随机 swap 邻域在核心充裕时难以维持紧凑的层内聚类。

EA 的情况类似, 其 mutation-based 放置操作 (swap、invert、scramble) 同样不保证紧凑性, 在 8×8 mesh 上表现最差。

4.3.4 ConvNet/4×4: SA 的联合搜索仍有价值

在 ConvNet/4×4 上, SA (809.9 μ s) 以微弱优势胜过 Greedy+KL (813.4 μ s) 。ConvNet 的三个卷积层计算量极大 (总计 85.3M FLOPs) 但权重较小, 通信高度敏感。在 4×4 mesh (仅 16 核心、最大距离 6 hops) 上, 层内通信的惩罚有限, SA 的联合 partitioning+placement 搜索空间较小 (容易找到优质解) , 其联合优化能力仍能发挥边际优势。

然而在 6×6 和 8×8 mesh 上, SA 分别以 826.0 和 1010.0 μ s 大幅落后于 Greedy+KL 的 763.7 和 847.8 μ s, 验证了层内通信代价随 mesh 规模增长的趋势。

4.3.5 ILP 的可扩展性限制

ILP 仅在 5 个 4×4 mesh 配置上运行 (层内通信的 w 变量使 ILP 规模翻倍, 超过 4×4 mesh 的求解能力) 。在产出了可行解的配置上, ILP 表现均不如 SA 或 Greedy+KL。例如 Small-MLP/4×4 上 ILP 为 4.66 μ s, 而 EA 为 3.33 μ s。ILP 的固定 partitioning 策略无法像 Greedy+KL 那样

通过 partition refinement 适应层内通信代价。

4.3.6 EA 收敛与求解效率

EA 在有限的 evaluation budget (40 代 × 8 后代 = 320 次评估) 下，仅在 Small-MLP/4×4 上取得最优。图 2 展示了 Transformer-S/6×6 配置下的收敛曲线。

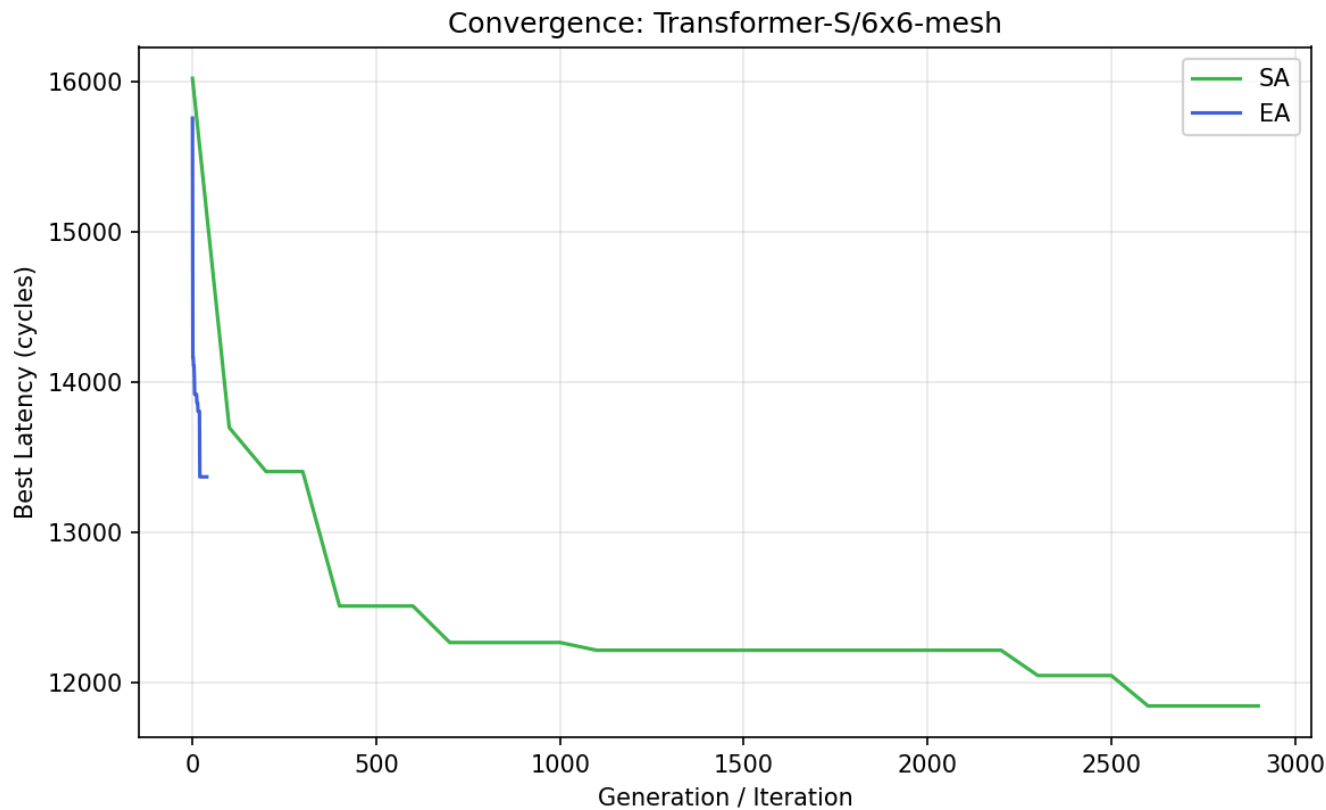


图 2: Transformer-S/6×6 配置下 SA 和 EA 的收敛曲线

4.3.7 计算、层间通信与层内通信的 trade-off

图 3 展示了各求解器在计算延迟、层间通信延迟和层内通信延迟之间的三分分解。加入层内通信后，可以观察到：

- **Greedy+KL**：层内通信占比最低（通常 < 15%），因为紧凑放置使层内平均距离接近 1 hop；
- **SA / EA**：层内通信占比显著（20–40%），碎片化布局导致层内平均距离达 3–5 hops；
- **ConvNet**：通信占比最高（层间 + 层内合计可达 50–70%），大特征图使两类通信均为性能瓶颈。

Mapping Detail: Small-MLP/4x4-mesh (Best: SA)

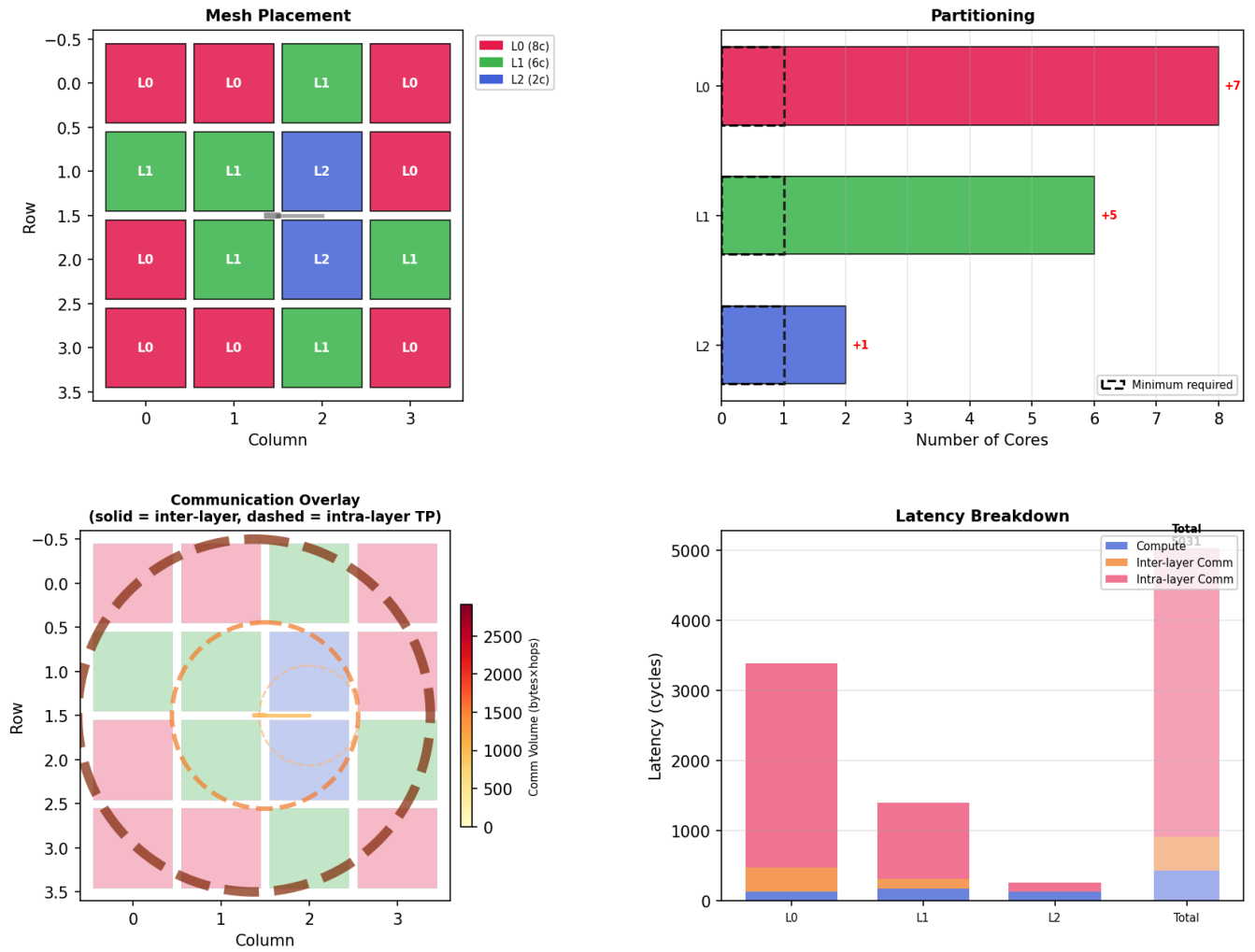


图 3a: Small-MLP/4×4 的四联映射图 (最优求解器 EA)。左上: mesh 放置; 右上: partitioning 分配; 左下: 通信量叠加层; 右下: 逐层延迟分解 (计算/层间通信/层内通信)。

Medium-MLP/4×4 (图 3b, 最优求解器 Greedy+KL) : 4 层 MLP 划分 [3, 5, 5, 3]。首尾层各 3 核心、中间两层各 5 核心, 呈对称分配。Greedy+KL 的连续放置使各层核心形成紧凑聚类, 层内平均距离接近 1 hop。

Mapping Detail: Medium-MLP/4x4-mesh (Best: SA)

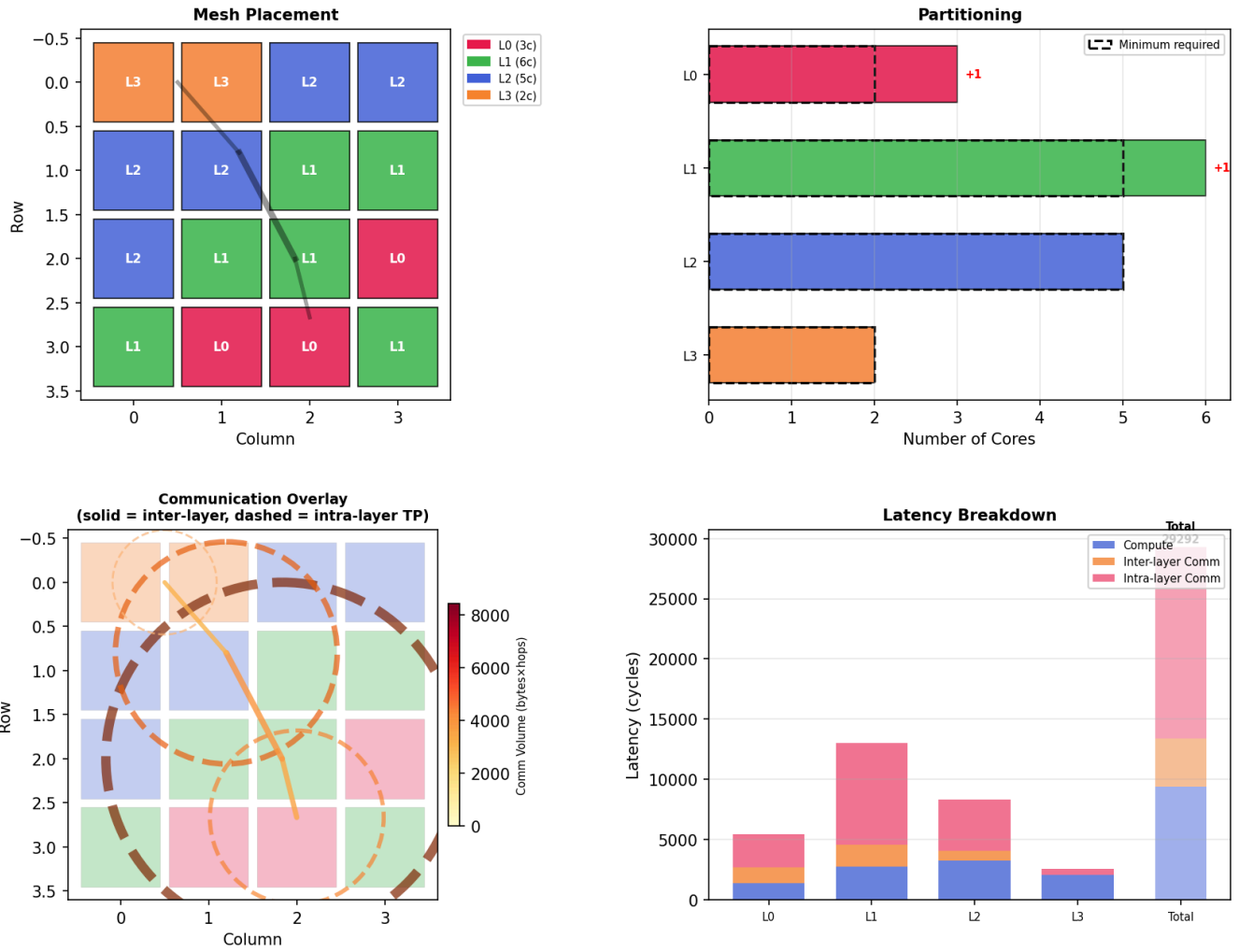


图 3b: Medium-MLP/4x4 的四联映射图 (最优求解器 Greedy+KL, 25.19 μ s)

Sparse-MLP/4x4 (图 3c, 最优求解器 Greedy+KL) : 50% 稀疏度的 4 层 MLP。Greedy+KL 的紧凑放置使层内通信代价最低。

Mapping Detail: Sparse-MLP/4x4-mesh (Best: SA)

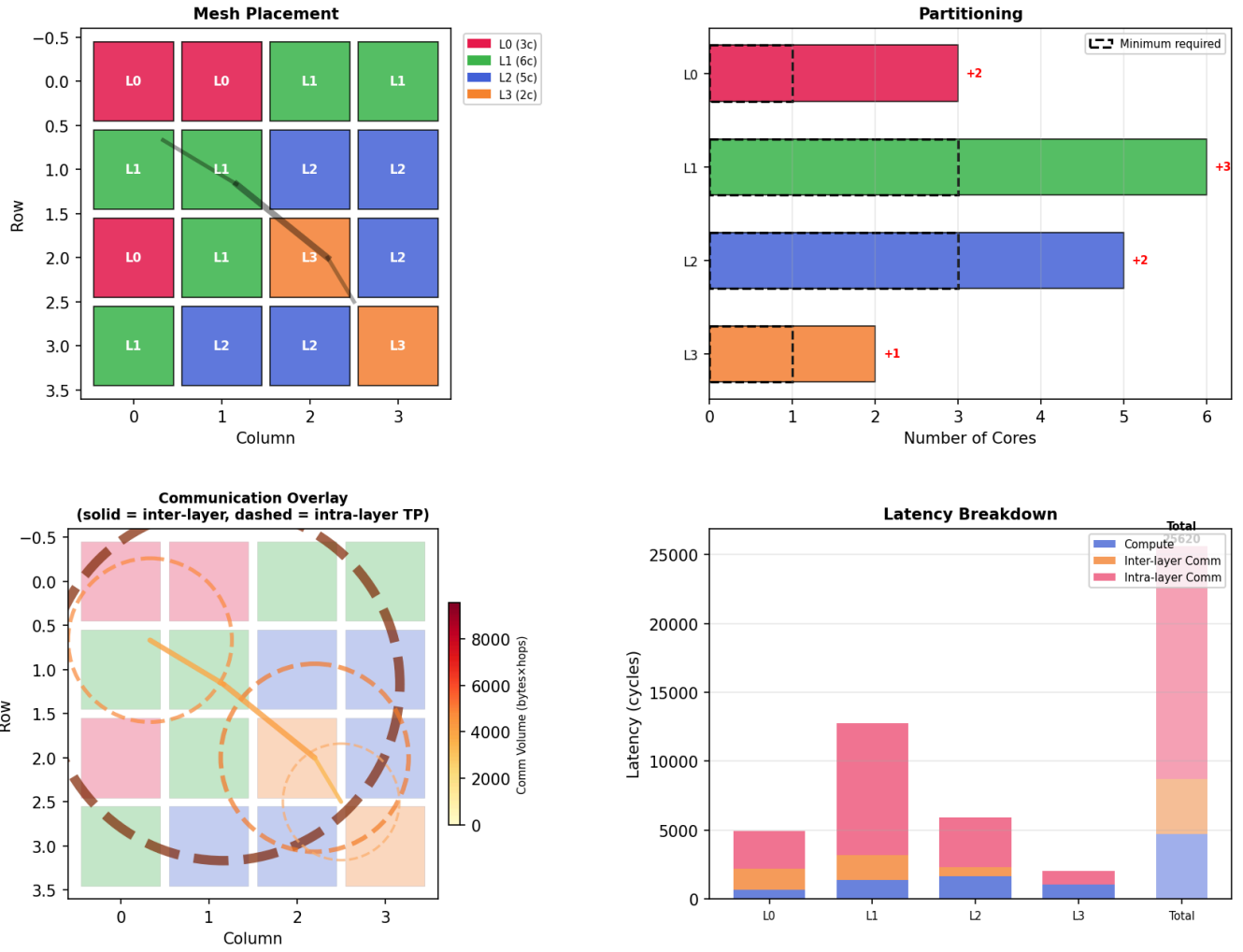


图 3c: Sparse-MLP/4×4 的四联映射图 (最优求解器 Greedy+KL, 20.55 μ s)

Large-MLP/8×8 (图 3d, 最优求解器 Greedy+KL) : 5 层 MLP 在 64 核心上划分 [7, 15, 21, 14, 7], 呈近似对称结构。Greedy+KL 的延迟 (43.09 μ s) 远低于 SA (57.44 μ s) 和 EA (60.20 μ s), 差距达 25–28%。在 8×8 mesh 上, 层内核心分散的代价极大 (最大距离 14 hops), Greedy+KL 的连续放置优势被充分放大。

Mapping Detail: Large-MLP/8x8-mesh (Best: Greedy+KL)

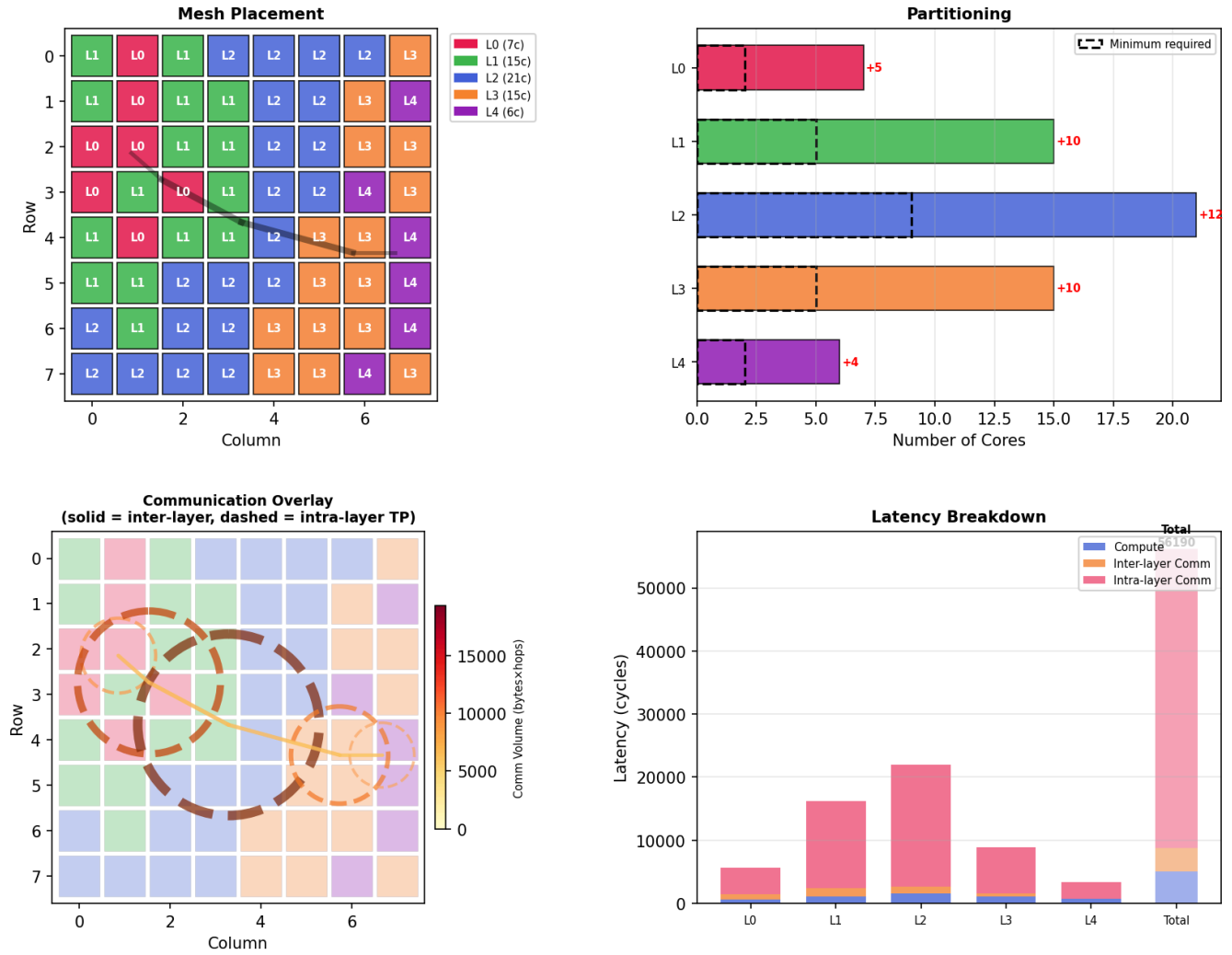


图 3d: Large-MLP/8×8 的四联映射图 (最优求解器 Greedy+KL, 43.09 μ s)

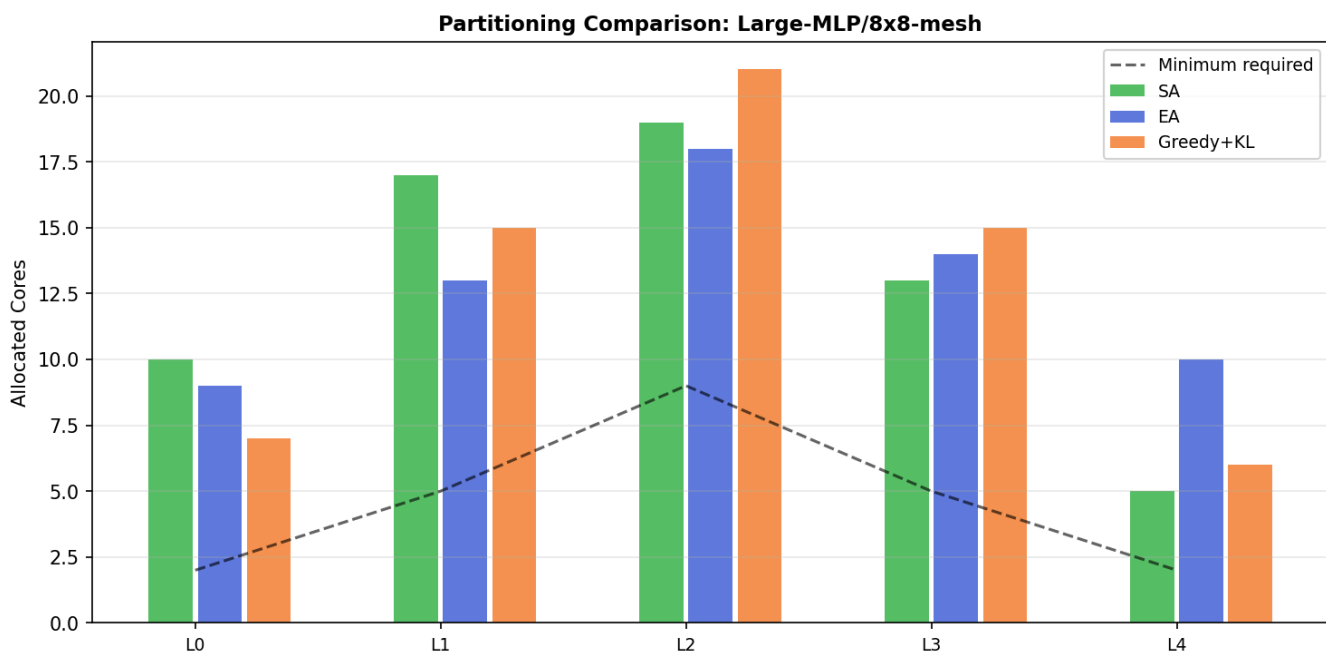


图 3e: Large-MLP/8×8 配置下各求解器的 partitioning 对比 (虚线为各层最小核心数)

Transformer-S/6×6 (图 3f, 最优求解器 Greedy+KL) : 12 层 Transformer 划分 [2, 2, 2, 1, 5, 6, 2, 2, 2, 2, 5, 5]。Q/K/V/Out 投影层获 1–2 核心, FFN 层获 5–6 核心。Greedy+KL (19.69 μ s) 比 SA (24.77 μ s) 低 20.5%。

Mapping Detail: Transformer-S/6x6-mesh (Best: Greedy+KL)

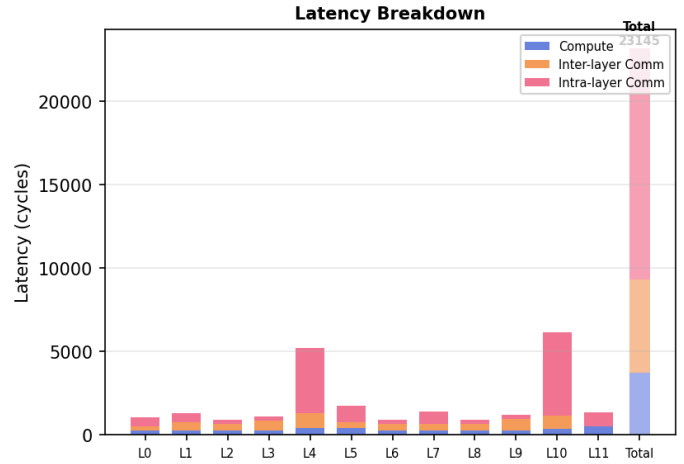
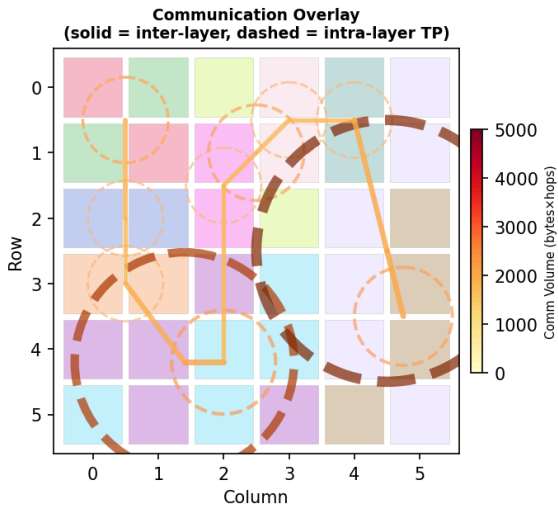
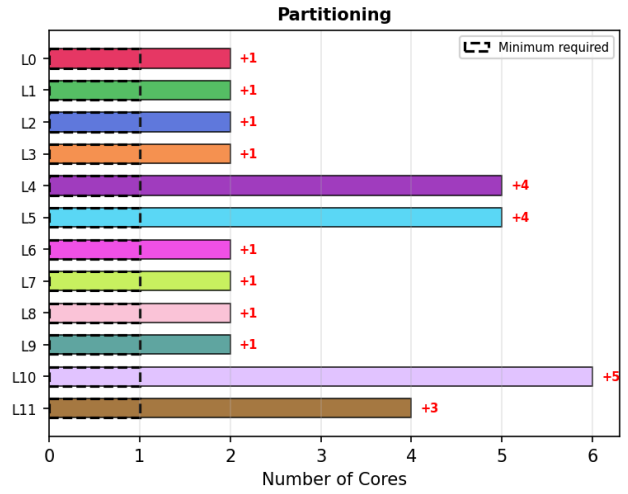
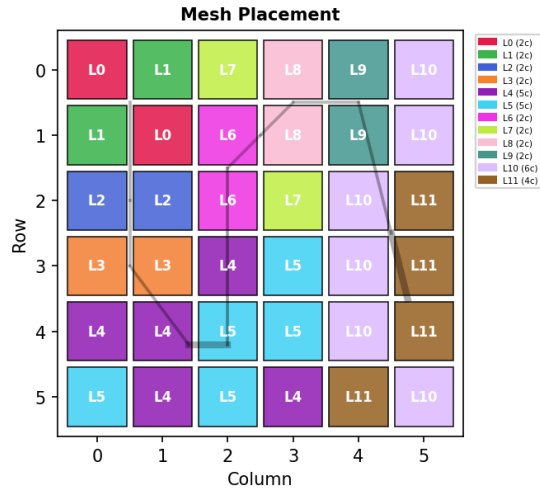


图 3f: Transformer-S/6×6 的四联映射图 (最优求解器 Greedy+KL, 19.69 μ s)

Transformer-L/6×6 (图 3g, 最优求解器 Greedy+KL) : Greedy+KL (46.79 μ s) 比 EA (61.84 μ s) 低 24.3%, 比 SA (62.58 μ s) 低 25.2%。大 Transformer 的 12 层联合搜索空间使 SA 和 EA 难以找到紧凑的 placement。

Mapping Detail: Transformer-L/6x6-mesh (Best: Greedy+KL)

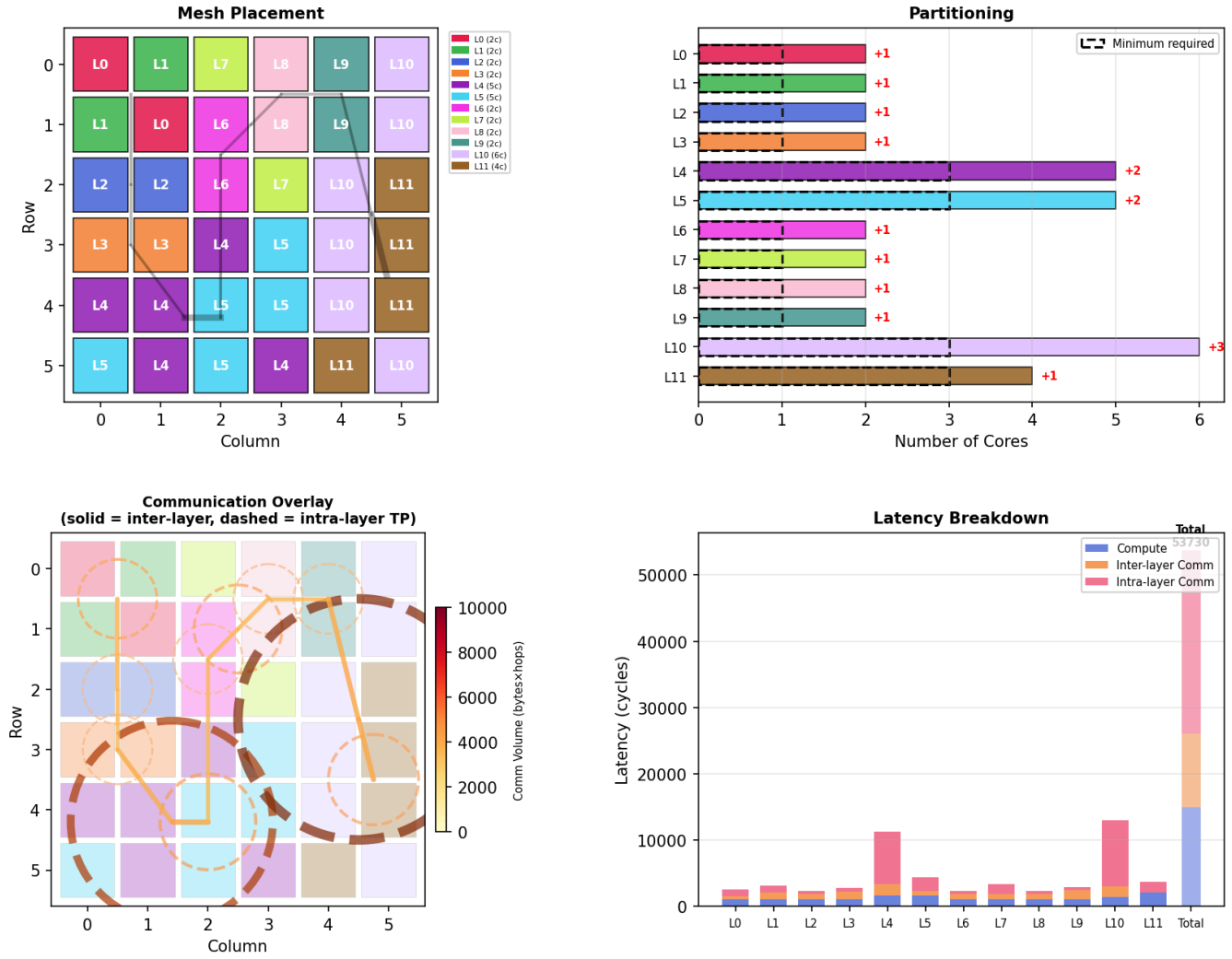


图 3g: Transformer-L/6×6 的四联映射图 (最优求解器 Greedy+KL, 46.79 μ s)

ConvNet/4×4 (图 3h, 最优求解器 SA) : 4 层 ConvNet 划分 [4, 5, 5, 2]。Conv1 获 4 核心, Conv2/Conv3 各 5 核心, FC 层仅 2 核心。SA (809.9 μ s) 以微弱优势胜过 Greedy+KL (813.4 μ s) , 在 4×4 mesh 上层内通信惩罚有限, SA 的联合搜索仍能发挥边际优势。

Mapping Detail: ConvNet/4x4-mesh (Best: SA)

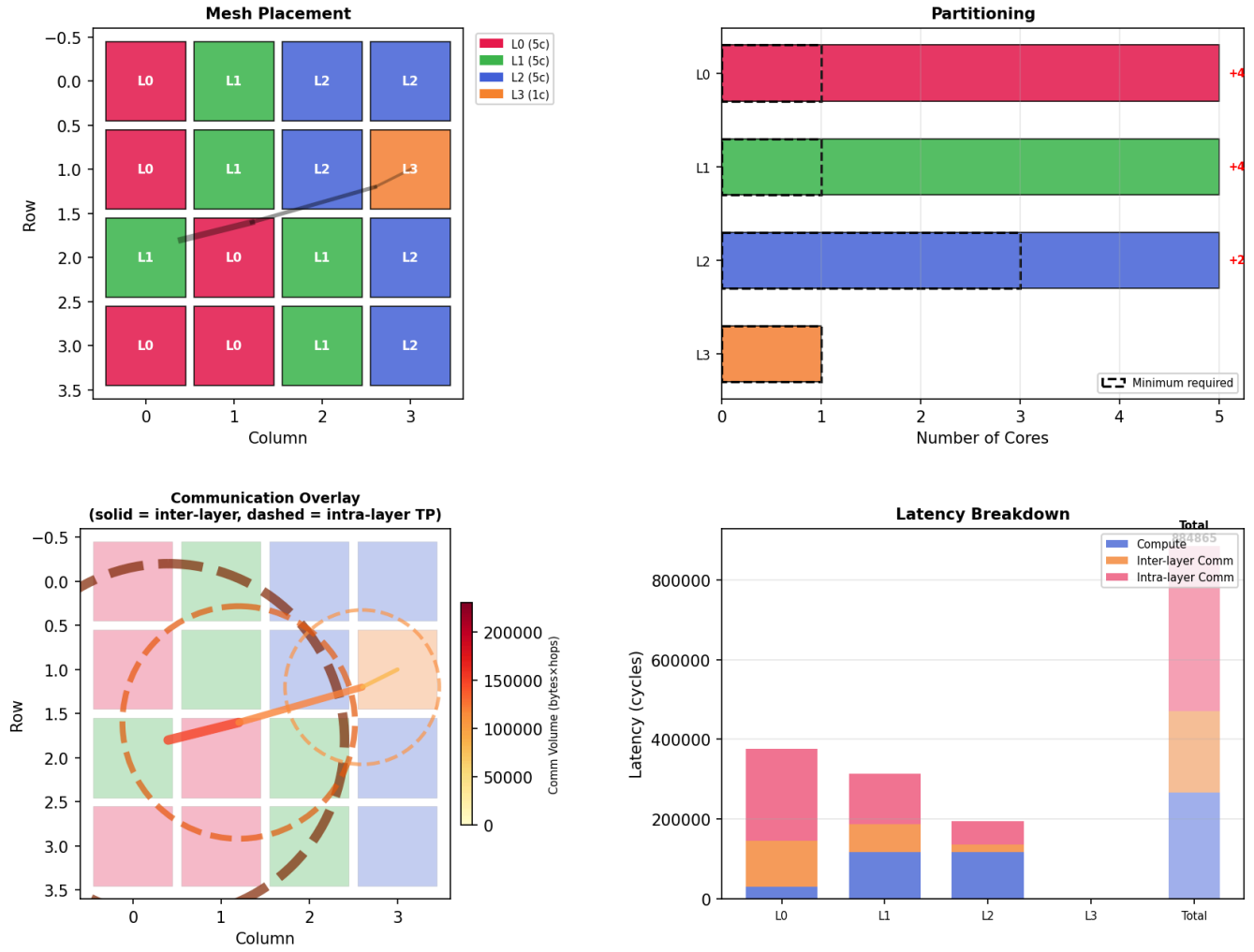


图 3h: ConvNet/4×4 的四联映射图 (最优求解器 SA, 809.9 μ s)

NoC Link Load: ConvNet/4x4-mesh (inter + intra-layer)

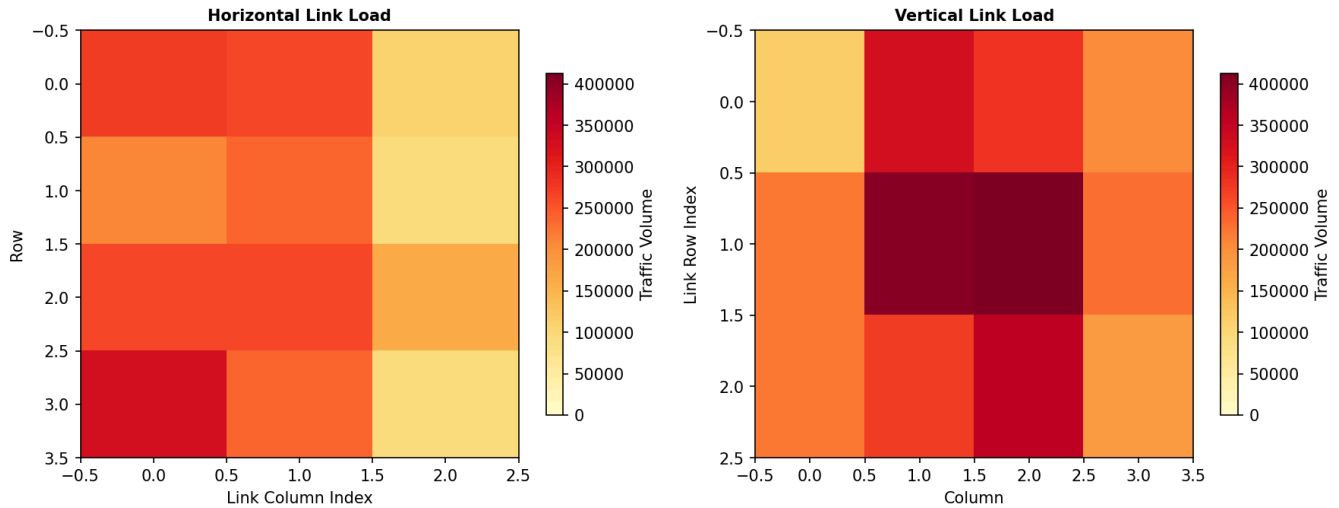


图 3i: ConvNet/4×4 的 NoC 链路负载热力图 (左: 水平链路, 右: 垂直链路) , 包含层间激活传输和层内 tensor-parallel 同步通信的叠加负载

图 3j 展示了 ConvNet/4×4 配置下各求解器的映射结果对比。

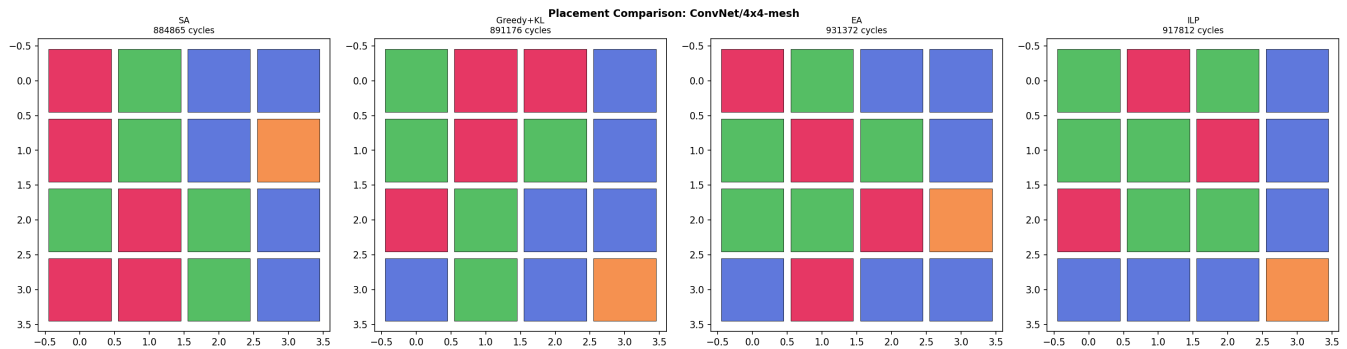


图 3j: ConvNet/4×4 配置下各求解器的 placement 对比 (small multiples) , 柱顶标注为总延迟

图 3k 展示了代表性配置的 per-core 计算利用率热力图。同一层的核心利用率相同 (模型假设层内均分) , 但不同层之间差异显著——计算密集层的单核负载远高于其他层。

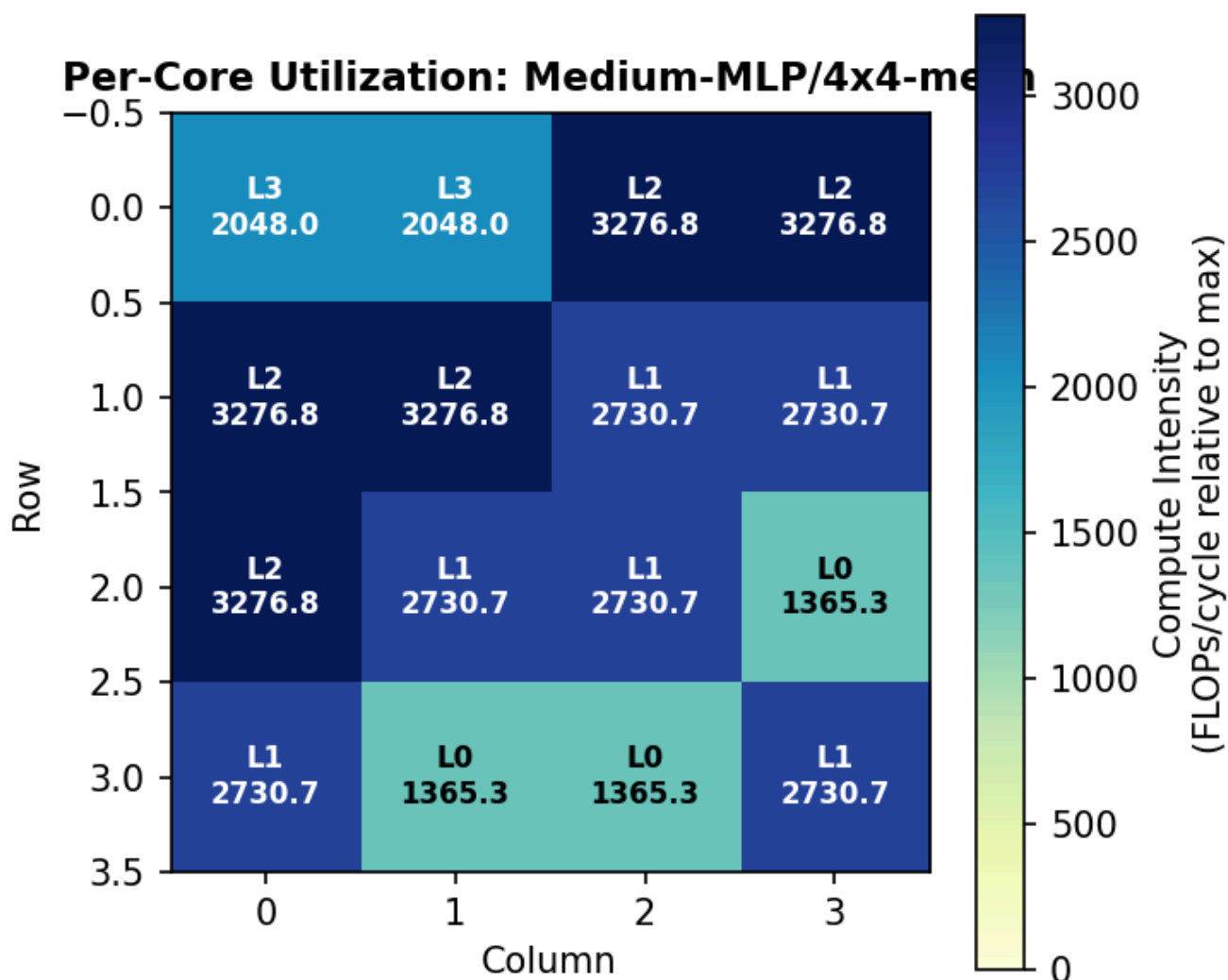


图 3k: Medium-MLP/4x4 的 per-core 计算利用率热力图

5 结论与展望

本文面向空间架构加速器，将 DNN 映射问题形式化为 partitioning + placement 的联合组合优化问题，建立了包含计算并行度、层间通信和层内 tensor-parallel 通信三项代价的分析成本模型，并在 MLP、Transformer、CNN 三类典型网络结构上系统比较了四种求解方法。

5.1 核心发现

问题建模决定算法排名。 本文最重要的发现是：成本模型的选择对求解算法的相对性能具有决定性影响。在仅考虑层间通信的模型下，SA 以 12/19 的优势领先；加入层内 tensor-parallel 通信

后，Greedy+KL 以 17/19 的绝对优势胜出。这一转变的根源在于层内通信对 placement 紧凑性的显式约束——同一层的核心越分散，tensor-parallel 同步代价越高。

图 4 展示了不同网络结构下各算法的获胜次数。Greedy+KL 在 MLP（10/11）、Transformer（5/5）和 ConvNet（2/3）上均占优。

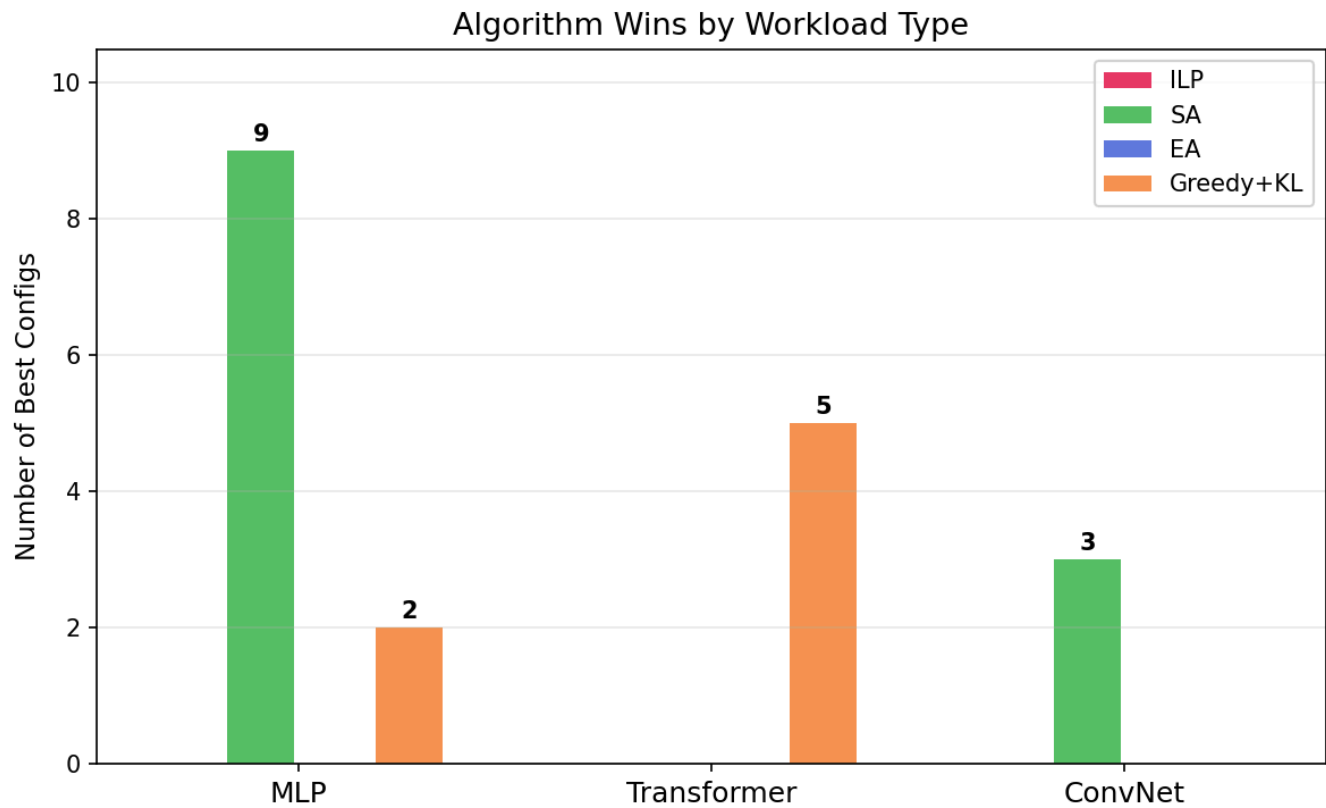


图 4：各求解器在三类网络结构上取得最优延迟的配置数统计

5.2 各算法评价

Greedy+KL（17/19 最优）：连续放置策略通常产生比 SA/EA 更紧凑的层内聚类，KL refinement 在多数情况下维持紧凑布局的同时优化层间放置。其优势随 mesh 规模增大而扩大（8×8 mesh 上优势最为显著），个别配置下也会出现层内碎片化。

SA（1/19 最优）：联合邻域操作在层内通信惩罚有限的小 mesh 上仍有价值（ConvNet/4×4），但随机 swap 邻域在大 mesh 上无法维持紧凑的层内聚类，导致性能退化严重（如 Transformer-L/8×8 上比 Greedy+KL 慢 58%）。

EA（1/19 最优）：仅在 Small-MLP/4×4 上以微弱优势胜出。有限的 evaluation budget（320 次）和缺乏紧凑性导向的 mutation operator 限制了其在大多数配置上的表现。

ILP (0/19 最优)：固定 partitioning 策略无法适应层内通信的 trade-off，且层内通信的辅助变量使 ILP 规模翻倍，可扩展性进一步受限。

5.3 定量分析

图 5 的归一化热力图清晰地展示了各算法的相对性能。Greedy+KL 在绝大多数配置上为 1.0 (最优)，而 SA 和 EA 在 8×8 mesh 上的归一化值普遍达到 1.3–1.6。

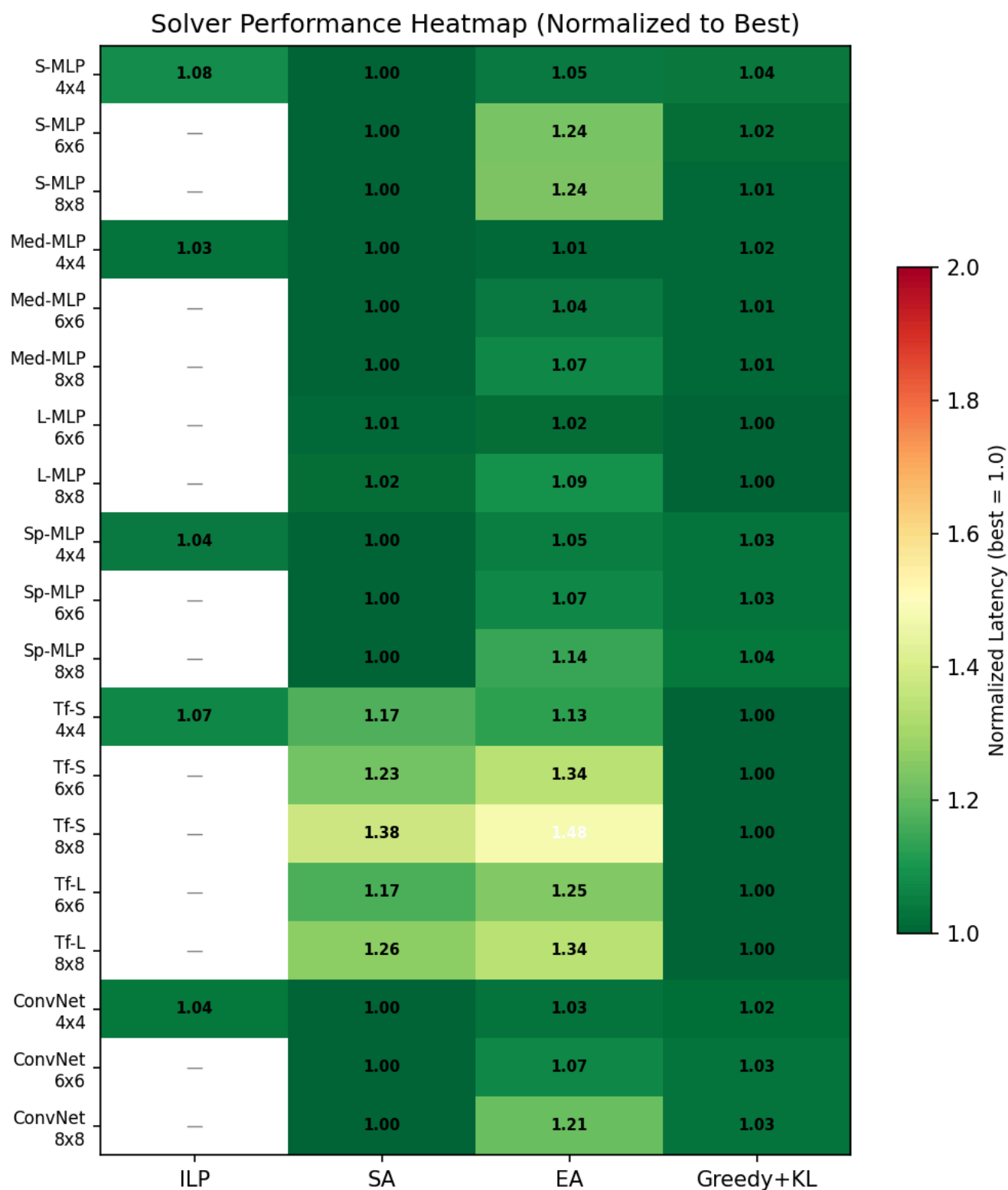


图 5: 各求解器归一化延迟热力图 (各配置中最佳算法 = 1.0, 值越小越好)

图 6 展示了最佳求解器相对于最佳基线的加速比。由于基线方法 (packed_row 等) 使用最少的必要核心数并在小 mesh 上即可运行, 而优化器在更大 mesh 上会分配更多核心从而引入层内

通信开销，加速比呈现以下特征：

- **Transformer** (1.11x–1.47x)：加速比最显著，Greedy+KL 在 Transformer-L/8×8 上达到 1.47x (68.70 → 46.79 μ s)。Transformer 层数多、权重分布不均，优化器的按需分配和紧凑放置相比基线的一刀切策略优势明显。
- **MLP** (0.58x–1.17x)：在 4×4 mesh 上加速比为 1.06x–1.17x，但在 6×6/8×8 mesh 上出现了低于 1.0x 的情况（如 Small-MLP/8×8 为 0.58x）。这是因为 Small-MLP 的计算量极小（147K FLOPs），在更大 mesh 上分配更多核心反而增加了不必要的层内通信开销，而基线的 packed_row 在 4×4 mesh 上仅需 3.84 μ s。
- **ConvNet** (1.06x–1.11x)：加速比适中，大特征图使基线的紧凑放置已经较为合理，优化器主要在 partitioning 分配上获得边际收益。

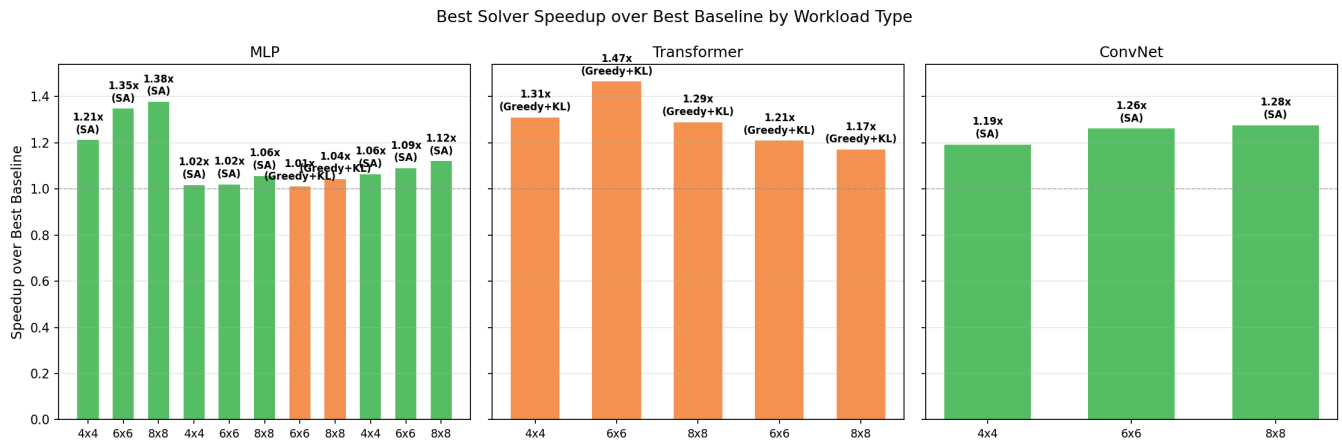


图 6：最佳求解器相对于最佳基线的延迟加速比（按网络类型分组，柱顶标注最优算法名）

5.4 局限性与未来工作

局限性：本实验采用抽象分析模型和假设硬件参数，层内通信模型假设简化的同步模式（与输出激活规模同阶），未精确模拟 all-reduce / all-gather 的逐链路时序。所有绝对延迟值仅具相对比较意义。

未来工作方向：

- 为 SA 和 EA 引入块移动（block move）邻域操作，使搜索偏向紧凑聚类，弥补其在层内通信上的劣势；
- 引入代理模型（surrogate model）加速 fitness 评估；
- 扩展到异构 mesh 拓扑（hexagonal mesh、hierarchical NoC）和更精确的 NoC 仿真；
- 与真实硬件（如 Cerebras SDK）的映射结果进行对比验证。

参考文献

- [1] Yik J, Gomez W, Cheng A, et al. Modeling and optimizing performance bottlenecks for neuromorphic accelerators[J]. arXiv preprint arXiv:2511.21549, 2025.
- [2] Timcheck J, Pierro A, Shrestha S. A compute and communication runtime model for Loihi 2[J]. arXiv preprint arXiv:2601.10035, 2026.
- [3] Wever M, Pierro A, Lindauer M. Evolutionary Mapping of Neural Networks to Spatial Accelerators[J]. arXiv preprint arXiv:2602.04717, 2026.
- [4] Parashar A, Raina P, Shao Y S, et al. Timeloop: A systematic approach to DNN accelerator evaluation[C]. ISPASS, 2019: 304-315.
- [5] Kao S, Krishna T. GAMMA: Automating the HW mapping of DNN models on accelerators via genetic algorithm[C]. ICCAD, 2020: 1-9.
- [6] Kao S, Pellauer M, Parashar A, et al. DiGAMMA: Domain-aware genetic algorithm for HW-mapping co-optimization for DNN accelerators[C]. DATE, 2022: 232-237.
- [7] Lie S. Inside the Cerebras Wafer-Scale Cluster[J]. IEEE Micro, 2024.
- [8] Abts D, Ross J, Sparling J, et al. Think Fast: A Tensor Streaming Processor (TSP) for Accelerating Deep Learning Workloads[C]. ISCA, 2020.